Session Cache

author: **LordSudo** · difficulty: **easy** · category: **memory forensics** · points: **100**

Challenge: a browser process memory capture. `memdump.bin` holds a cache of session records somewhere inside, and the goal is to retrieve them. A short `FORMAT_SPEC.txt` ships with it.

1. Inspect the archive

List first, then extract. The `__MACOSX` entries are resource-fork junk and can be ignored.

kali@kali: ~/ctf/Session_Cache

```
┌──(enos㉿kali)-[~/ctf]
└─$ unzip -l Session_Cache.zip
Archive:  Session_Cache.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-19 16:49   Session_Cache/
      820  2026-10-19 16:49   Session_Cache/FORMAT_SPEC.txt
  5242880  2026-09-20 05:40   Session_Cache/memdump.bin
---------                     -------
                              5 files

┌──(enos㉿kali)-[~/ctf]
└─$ unzip -q Session_Cache.zip && cd Session_Cache

┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ file *
FORMAT_SPEC.txt: ASCII text
memdump.bin:     data
```

2. Read the format spec

kali@kali: ~/ctf/Session_Cache

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ cat FORMAT_SPEC.txt
Format spec:

  Look for the marker "SESSCACHE_v2" in the dump. Immediately after
  it: a 4-byte little-endian record count, then that many fixed-size
  records. Each record holds a plain record_id, a plain timestamp, an
  obfuscated username, an obfuscated session_token, and a trailing
  plain checksum (the sum of the decoded username+session_token bytes,
  mod 2^32) ...

  The obfuscated region of each record is XOR'd against a keystream
  derived from that record's own record_id:

    key_byte(record_id, i) = ((record_id * 2654435761 + i * 40503) & 0xFFFFFFFF) & 0xFF
```

So the plan is: find the marker, work out the record layout, rebuild the keystream per record, XOR, and use the checksum to confirm.

3. Try the obvious: strings

kali@kali: ~/ctf/Session_Cache

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ strings -n 6 memdump.bin | head -5
=%USSY
L|hu    vx
qNF%dD
:y &92
7,Drk*

┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ strings -n 6 memdump.bin | grep -iE 'sess|r00t|flag'
(only SESSCACHE_v2 appears; nothing readable)
```

The dump is mostly random noise and the records are obfuscated, so plain `strings` finds nothing useful. That confirms the XOR step is required.

4. Locate the marker

kali@kali: ~/ctf/Session_Cache

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ grep -obUaP 'SESSCACHE_v2' memdump.bin
2234639:SESSCACHE_v2

┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ dd if=memdump.bin bs=1 skip=2234639 count=64 2>/dev/null | hexdump -C
00000000  53 45 53 53 43 41 43 48  45 5f 76 32 00 00 00 00  |SESSCACHE_v2....|
00000010  12 00 00 00 e8 03 00 00  80 0e 80 69 00 e6 b7 60  |...........i...`|
00000020  25 16 dd 9d 4f 57 8e c5  fc 33 6a a1 ab 6a 35 0e  |%...OW...3j..j5.|
00000030  eb 8d 40 6a a5 f5 c6 0d  0d c2 ea 20 7f 1e d4 d8  |..@j... ... ....|
```

One hit, at offset `2234639`. The marker is 12 bytes. Four zero bytes follow, then `0x12 = 18`, which is the record count. The spec says the count comes "immediately after" the marker, but reading it directly at +12 gives 0. The real count sits behind a 4-byte pad, so records start at `marker + 12 + 8`.

5. Work out the record size

The first record starts with `e8 03 00 00` = 1000. Scanning for nearby ids shows where the next records begin.

kali@kali: ~/ctf/Session_Cache — python3

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ python3 - <<'E'
import struct
d=open('memdump.bin','rb').read()
base=2234639+12+8
for rid in (1000,1009,1017):
    p=d.find(struct.pack('<I',rid),base,base+2000)
    print(rid,p-base)
E
1000 0
1009 76
1017 152
```

The record length is `76 bytes`. Ids are not sequential (1000, 1009, 1017, ...), which is fine.

offset	size	field
0	4	record_id (u32 LE)
4	4	timestamp (u32 LE)
8	32	username (XOR'd)
40	32	session_token (XOR'd)
72	4	checksum (u32 LE)

The obfuscated region is 64 bytes (offsets 8–71), and the keystream index `i` runs 0–63 across it.

6. Decode every record and verify checksums

kali@kali: ~/ctf/Session_Cache — solve.py

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ cat solve.py
import struct
d = open('memdump.bin','rb').read()
base = d.find(b'SESSCACHE_v2') + 12 + 8      # marker + pad + count
n = struct.unpack('<I', d[base-4:base])[0]    # 18

def dec(rid, b):
    return bytes(x ^ (((rid*2654435761 + i*40503) & 0xFFFFFFFF) & 0xFF)
                 for i, x in enumerate(b))

for k in range(n):
    r = d[base+76*k : base+76*(k+1)]
    rid, ts = struct.unpack('<II', r[:8])
    ck = struct.unpack('<I', r[72:])[0]
    p = dec(rid, r[8:72])
    user  = p[:32].rstrip(b'\0').decode()
    token = p[32:].rstrip(b'\0').decode()
    print(rid, ts, sum(p) & 0xFFFFFFFF == ck, user, token)

┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ python3 solve.py
1000 1770000000 True hyamamoto  sess_fb35288aa017ab5a68807255866c4fd1
1009 1770000300 True rpatel     sess_41e3584df7162c667fcd364e996a1f5d
1017 1770000600 True tnguyen    sess_6ba3800665b6398a7e6283393144de5b
1023 1770000900 True hyamamoto  sess_a6fb49f3e221fb032bfb788c0bb70e20
1030 1770001200 True rpatel     sess_e331c27d305122a40a06987f544dfbfd
1037 1770001500 True kwilliams  sess_17b66f40ab82ab4ac11500b1ee085c54
1043 1770001800 True kwilliams  sess_9056859370bc388f18cec754c29c0595
1049 1770002100 True hyamamoto  sess_4f2b5942be72f3ba920e9838344df6e0
1058 1770002400 True dcohen     sess_261b0f55af3a1b593f583fa139e5a835
1066 1770002700 True sokafor    sess_27e3689ad1c6d1f18742f90d9b386c47
1073 1770003000 True asingh     sess_d2d24f550731896c17006b13e8477cc6
1077 1770003300 True fibrahim   r00t{x0r_k3ys_d3r1v3d_fr0m_1ds_4r3nt_3n0ugh}
1086 1770003600 True dcohen     sess_344a9b7f724488fd2b374e2865bc1dc8
1091 1770003900 True jmartinez  sess_6a6e965052432a0a0e2adb21978793f5
1100 1770004200 True sokafor    sess_339f2b75b8b6dd60a59a5f848ace7958
1108 1770004500 True bchen      sess_c9cfd7c6c0f6f3f3bb4e11ad2f27d9da
1113 1770004800 True mrossi     sess_72da1e427e03c6d2e5484abe00dd4831
1122 1770005100 True bchen      sess_b7e4904928798f2955a04f588d0b642a
```

All 18 checksums match, so the layout and keystream are correct. Seventeen tokens look like `sess_<32 hex>`. Record 1077 (user `fibrahim`) breaks the pattern and holds the flag.

7. Flag

kali@kali: ~/ctf/Session_Cache

```
┌──(enos㉿kali)-[~/ctf/Session_Cache]
└─$ python3 solve.py | grep -o 'r00t{[^}]*}'
r00t{x0r_k3ys_d3r1v3d_fr0m_1ds_4r3nt_3n0ugh}
```

Flag: r00t{x0r_k3ys_d3r1v3d_fr0m_1ds_4r3nt_3n0ugh}

Key takeaways

- **Anchor on a marker.** Large memory dumps are mostly noise. A known string like `SESSCACHE_v2` found with `grep -obUaP` gives a byte offset to start from.
- **Specs can be incomplete.** The count was not directly after the marker; a 4-byte pad sat in between. Always dump the raw bytes with `hexdump` and check them against the description.
- **Recover fixed record sizes from repeated fields.** Searching for known ids (1000, 1009, 1017) and measuring the gaps gave the 76-byte stride.
- **Use the built-in checksum.** 18 of 18 matches proved the offsets, field sizes and keystream were right, with no guessing.
- **A keystream derived from public data is not encryption.** The key was computed from the plaintext `record_id` stored beside the ciphertext, so anyone with the dump can decode it. That is the point of the flag text.
- **Hunt for the outlier.** Seventeen of eighteen tokens followed `sess_<hex>`; the one that did not was the flag.